Offchain Labs Arbitrum ArbOS 60/61

Security Assessment (Summary Report)

July 10, 2026

Prepared for:
Harry Kalodner, Steven Goldfeder, and Ed Felten
Offchain Labs

Prepared by: **Jaime Iglesias and Simone Monica**

Table of Contents

Project Summary

Contact Information

The following project manager was associated with this project:

Mary O'Brien, Project Manager
mary.obrien@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Jaime Iglesias, Consultant
jaime.iglesias@trailofbits.com

Simone Monica, Consultant
simone.monica@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
February 24, 2026	Pre-project kickoff call
July 2, 2026	Delivery of final report draft
July 10, 2026	Delivery of final report (including payload)

Project Targets

The engagement involved reviewing and testing the targets listed below.

Nitro

Repository	https://github.com/OffchainLabs/nitro
Version	746bda2febea6b160f56267f0727cf0bc21e9eb7 26d4dc21f9597fcc345c5ff1eb3355eb7444cae4 Fd2a5b01b24c18c21c6d6f3748f7fd887859bc44 ec997e665e5851a0e0976eb1e169b39fae532fff 3569036ece4dc0a95f3f1926165cdf33ca3456dc d162cc12e8c30db9248b41e55e40450d2b4da9b8 C9fbd05597ad51258612039dca9348ab7d03d4ee
Type	Go, Rust
Platform	Arbitrum

Go-Ethereum

Repository	https://github.com/OffchainLabs/go-ethereum
Version	c59dcce8b117bcee69a792f278fd9412f0586048 fa29c9c651a28a8f91d698a9b45e7da86ec585c9 9c369aa02836740d2844d037721e2fba6d573328 f1c5f02b29e7c499232ecbd17bc0e32801f8181c
Type	Go
Platform	Arbitrum

Nitro-Contracts

Repository	https://github.com/OffchainLabs/nitro-contracts
Version	PR#415 PR#416 PR#421 PR#423 PR#428 PR#429
Type	Solidity

Platform	Arbitrum

Governance

Repository	https://github.com/ArbitrumFoundation/governance
Version	PR#374
Type	Solidity
Platform	Arbitrum

Executive Summary

Engagement Overview

Offchain Labs engaged Trail of Bits to review the security of Nitro and its `go-ethereum` fork, particularly of the ArbOS 60 and 61 upgrades.

A team of two consultants conducted the review from February 24 to March 16, 2026. The review then continued from March 23 to April 13, 2026, and later continued from April 27, 2026 to May 8, 2026, with the final commit reviewed from June 3, 2026 to June 9, 2026, for a total of 24 engineer-weeks of effort.

Finally, on July 9, 2026, the ArbOS 61 upgrade payload was reviewed.

With full access to source code and documentation, we performed static and dynamic testing of the targets, using automated and manual processes.

Observations and Impact

ArbOS 60 and 61 are the next Nitro upgrades, designed to bring several new features to Nitro, including:

- Stylus program's increased code size through program fragments
- Multi-dimensional constraint gas-pricing model
- Multiple bug fixes

The main focus of the review was to determine whether these features were correctly implemented, identifying potential edge cases and missing functionality.

Additionally, we looked for potential consensus-breaking issues, logical issues, issues related to Arbitrum-specific behavior, and any unintended consequences arising from the new features.

Most of the issues we identified are informational in severity and related to edge cases. The high-severity issue concerns a mapping iteration that is non-deterministic in Go.

Recommendations

- **Remediate the findings disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Non-deterministic map iteration in multi-gas constraint precompiles	Undefined Behavior	High
2	Gas is charged after state read	Undefined Behavior	Informational
3	Inconsistent write-protection on ArbOS state handle in block production	Undefined Behavior	Informational
4	RevertToSnapshot after Finalise causes a panic in extraPostTxFilter error path	Undefined Behavior	Informational
5	Stale per-dimension base fees are reused when multi-gas constraints are re-enabled	Data Validation	Informational
6	The initial-backlog cap can be bypassed by under-supplying gas to the constraint validation	Data Validation	Informational
7	The base-fee cap exponent delivers ~365x instead of the documented ~5,000x	Undefined Behavior	Informational
8	A SingleDim resource weight is accepted and can under-prices every other dimension	Data Validation	Informational
9	Saturating multiplication collapses the constraint exponent and bypasses the initial-backlog cap	Data Validation	Informational

Detailed Findings

1. Non-deterministic map iteration in multi-gas constraint precompiles

Severity: **High**	Difficulty: **Low**
Type: Undefined Behavior	Finding ID: TOB-ARBOS60-1
Target: `precompiles/ArbGasInfo.go`, `arbos/l2pricing/multi_gas_constraint.go`	

Description

The `GetMultiGasPricingConstraints` function in the `ArbGasInfo` precompile iterates over a Go map to build the `resources` slice for each constraint. Because Go map iteration order is non-deterministic, the resulting `WeightedResource` array can have its elements in a different order on different nodes. The slice is ABI-encoded and returned to callers, so any on-chain consumer (i.e., a smart contract calling this precompile) will receive return data that varies across validators, which can lead to a consensus failure.

The function calls `constraint.ResourcesWithWeights()`, which returns a `map[multigas.ResourceKind]uint64`. It then ranges over this map to populate a `[]WeightedResource` slice:

```
resourceMap, err := constraint.ResourcesWithWeights()
if err != nil {
    return nil, fmt.Errorf("failed to read resource weights for constraint %d: %w",
i, err)
}


resources := make([]WeightedResource, 0, len(resourceMap))
for kind, weight := range resourceMap {
    resources = append(resources, WeightedResource{
        Resource: uint8(kind),
        Weight:   weight,
    })
}
```

Figure 1.1: Non-deterministic map iteration populates the resources slice (nitro/precompiles/ArbGasInfo.go#L354–L365)

No sorting step follows the loop, so the order of elements in `resources` depends entirely on the runtime's map iteration randomization.

A similar pattern exists in `SetResourceWeights` in `MultiGasConstraint`. The function iterates over a `map[uint8]uint64` to validate resource kinds:

```
func (c *MultiGasConstraint) SetResourceWeights(weights map[uint8]uint64) error {
    var maxWeight uint64
    for kind, weight := range weights {
        if _, err := multigas.CheckResourceKind(kind); err != nil {
            return err
        }
        // ...
    }
    // ...
}
```

Figure 1.2: Non-deterministic map iteration in validation loop (nitro/arbos/l2pricing/multi_gas_constraint.go#L77–L86)

If the map contains multiple invalid resource kinds, the error returned on the first iteration will vary across nodes, since map iteration order is randomized. However, since precompile errors of this type are not encoded into revert data (the error string is discarded and a generic `ErrExecutionReverted` is returned), this instance does not currently affect consensus. It remains a correctness concern and a latent risk if error handling changes in the future.

Exploit Scenario

A smart contract calls `ArbGasInfo.GetMultiGasPricingConstraints()` and uses the returned array to make a state-changing decision, such as selecting a resource type based on its position in the array. Because the order of `WeightedResource` elements within each constraint differs across validator nodes, the contract's execution diverges: one validator commits a different state transition than another. This inconsistency prevents the network from reaching consensus, potentially halting the chain or causing a fork.

Recommendations

Short term, sort the `resources` slice by `Resource` (the resource kind ID) before appending it to the result in `GetMultiGasPricingConstraints`. Similarly, in `SetResourceWeights`, iterate over sorted keys rather than ranging directly over the map. This ensures deterministic ordering of both return data and error messages across all nodes.

Long term, adopt a project-wide convention that prohibits ranging over maps in any consensus-critical code path. Consider adding a linter rule or code review checklist item that flags `for ... range` over map types in packages under `precompiles/`, `arbos/`, and

any other locations whose code is part of the state transition. This prevents future introductions of the same class of non-determinism.

2. Gas is charged after state read

Severity: **Informational**	Difficulty: **Low**
Type: Undefined Behavior	Finding ID: TOB-ARBOS60-2
Target: `arbos/programs/programs.go`	

Description

ArbOS60 introduces Stylus root programs, which effectively increase the effective deployable size of Stylus programs by allowing a compressed WASM to be split across multiple contracts (fragments).

These fragments are reconstructed (merged) and decompressed during activation of the root contract, replicating the behavior of previous Stylus versions.

As shown in figure 1.1, once the Stylus root is fetched, the fragment array is iterated over to fetch the pieces of the compressed WASM. Because these fragments are split across multiple contracts, they must be read during activation, so extra gas is required to account for this additional work.

Note that gas charging is needed only during activation. Once a Stylus program is activated, its binary lives in the WASM store, so there is no need to constantly decompress a program when it is called.

```
func getWasmFromRootStylus(statedb vm.StateDB, data []byte, maxSize uint32,
maxFragments uint8, burner burn.Burner) ([]byte, error) {
      root, err := state.NewStylusRoot(data)
      if err != nil {
         return nil, err
      }

      [...]

       var compressedWasm []byte
       for _, addr := range root.Addresses {
              fragCode := statedb.GetCode(addr)
              if burner != nil {
                     if err := chargeFragmentReadGas(burner, statedb, addr,
uint64(len(fragCode))); err != nil {
                            return nil, err
                     }
              }
```

Figure 2.1: Part of the `getWasmFromRootStylus` function (`programs.go#358–L409`)

However, as shown above, the `GetCode` function on the statedb is first called to retrieve the code from the fragment's address, and later the actual gas is calculated and charged for that code.

This is a problem when the fragment's address is cold (i.e., it is not in the access list), since this requires retrieving the account from state, which is an expensive operation. An attacker could initiate an activation while sending a very small amount of gas, forcing the node to perform a cold read of the fragment, then revert once it tries to charge the gas for the read.

The impact of this issue is very limited: first, it only occurs during a cold read (warm reads are "cheap"), and second, an attacker would need to deploy the Stylus program and then try to activate it; activation pre-emptively charges a steep amount of gas, so the cost for the attacker is already high.

Note that this issue was addressed by PR#4489; users are now required to send enough gas for at least a worst-case code read (i.e., the max code). If they do not have sufficient gas, then the state read never happens, mitigating the issue.

Recommendations

Short term, ensure the user has enough gas to perform the state read. Note that this requires either knowing the code size in advance (e.g., by storing the size for each fragment in the root) or making an assumption about it (e.g., that it is a worst-case read).

Long term, thoroughly document the solution.

3. Inconsistent write-protection on ArbOS state handle in block production

Severity: **Informational**	Difficulty: **High**
Type: Undefined Behavior	Finding ID: TOB-ARBOS60-3
Target: `arbos/block_processor.go`	

Description

The `ProduceBlockAdvanced` function opens an ArbOS state handle with write access (`readOnly=false`) at block-production entry, but two subsequent re-open sites within the same function use `readOnly=true`. This creates an inconsistent write-protection invariant on the state handle that persists throughout block processing. While the inconsistency does not currently lead to exploitable behavior, it introduces a fragile contract that future code changes could silently violate.

The `readOnly` flag propagates through the `SystemBurner` into `Storage.Set()` and `StorageSlot.Set()`, which reject writes with `vm.ErrWriteProtection` when the burner is read-only:

```
func (s *Storage) Set(key common.Hash, value common.Hash) error {
	if s.burner.ReadOnly() {
		log.Error("Read-only burner attempted to mutate state", "key", key,
"value", value)
		return vm.ErrWriteProtection
	}
	// ...
```

Figure 3.1: Write-protection check in storage operations (nitro/arbos/storage/storage.go#L183–L189)

At the start of `ProduceBlockAdvanced`, the state is opened with `readOnly=false` to allow `CommitMultiGasFees()` to write next-block fee data into the current-block storage slots:

```
arbState, err := arbosState.OpenSystemArbosState(statedb, nil, false)
if err != nil {
	return nil, nil, nil, err
}
// ...
l2Pricing := arbState.L2PricingState()
err = l2Pricing.CommitMultiGasFees()
```

Figure 3.2: Non-deterministic map iteration in validation loop (nitro/arbos/block_processor.go#L306–L326)

However, after the `StartBlock` internal transaction (which runs as the first transaction in every block), the handle is re-opened with `readOnly=true`:

```
if tx.Type() == types.ArbitrumInternalTxType {
	// ArbOS might have upgraded to a new version, so we need to refresh our state
	buildState.arbState, err = arbosState.OpenSystemArbosState(buildState.statedb,
nil, true)
	// ...
}
```

Figure 3.3: State handle re-opened as read-only after internal transaction (nitro/arbos/block_processor.go#L552–L562)

The same `readOnly=true` re-open occurs in `rollbackToGroupCheckpoint`, which runs when a cascading redeem is filtered and the transaction group is reverted:

```
func (s *blockBuildState) rollbackToGroupCheckpoint(header *types.Header) error {
	cp := s.activeGroupCP
	cp.backup.RevertToSnapshot(cp.snap)
	s.statedb = cp.backup
	// ...
	s.arbState, err = arbosState.OpenSystemArbosState(s.statedb, nil, true)
	// ...
}
```

Figure 3.4: State handle re-opened as read-only after internal transaction (nitro/arbos/block_processor.go#L126–L144)

As a result, the `arbState` handle passed to sequencing hooks (`PreTxFilter`, `PostTxFilter`, `extraPostTxFilter`) carries different write permissions depending on when it is accessed during block processing. The writable window is short-lived (active only during `CommitMultiGasFees` and the internal transaction), after which all user transactions see a read-only handle. If future changes introduce a write through the state handle after either re-open site, that write would fail with `ErrWriteProtection`. Because `Restrict()` on the system burner only logs the error rather than panicking, such a failure could produce silent state divergence across nodes rather than a visible crash.

Recommendations

Short term, scope the writable state handle exclusively to the `CommitMultiGasFees` call by opening a separate writable `ArbosState` for that operation and keeping the main block-production handle read-only (`readOnly=true`) at all three open sites. This ensures a uniform write-protection invariant throughout block processing and prevents future mutations from accidentally bypassing or being blocked by inconsistent read-only settings.

Long term, document how the ArbOS state should be opened during the different phases of block production (i.e., read-only or not), and consider adding additional tests to catch future changes that could potentially break the invariant.

4. RevertToSnapshot after Finalise causes a panic in extraPostTxFilter error path

Severity: **Informational**	Difficulty: **Low**
Type: Undefined Behavior	Finding ID: TOB-ARBOS60-4
Target: `arbos/block_processor.go`, `arbos/extra_transaction_checks.go`	

Description

The `extraPostTxFilter` hook in the block production loop calls `RevertToSnapshot` on a snapshot ID that has already been invalidated by `Finalise`, causing a guaranteed panic if the filter ever returns an error.

During block production, `ProduceBlockAdvanced` creates a state snapshot before applying each transaction, then calls `ApplyTransactionWithResultFilter` to execute the transaction. Inside `ApplyTransactionWithResultFilter`, after the result filter callback succeeds, `Finalise` is called on the `StateDB`. `Finalise` clears the journal via `clearJournalAndRefund`, which calls `journal.reset()`, wiping all valid revision IDs.

```
snap := buildState.statedb.Snapshot()
buildState.statedb.SetTxContext(tx.Hash(), len(buildState.receipts))


gasPool := gethGas
blockContext := core.NewEVMBlockContext(header, chainContext, &header.Coinbase)
evm := vm.NewEVM(blockContext, buildState.statedb, chainConfig,
vm.Config{ExposeMultiGas: exposeMultiGas})
receipt, result, err := core.ApplyTransactionWithResultFilter(
    evm,
    &gasPool,
    buildState.statedb,
    header,
    tx,
    &header.GasUsed,
    runCtx,
    func(result *core.ExecutionResult) error {
        // ...
        return nil
    },
)
if err != nil {
    // This path is safe: Finalise has not yet run
    buildState.statedb.RevertToSnapshot(snap)
```

```
    buildState.statedb.ClearTxFilter()
    return nil, nil, err
}

// Additional post-transaction validity check
if err = extraPostTxFilter(chainConfig, header, buildState.statedb,
buildState.arbState, tx, options, sender, l1Info, result); err != nil {
    buildState.statedb.RevertToSnapshot(snap) // PANIC: snap is invalid
    buildState.statedb.ClearTxFilter()
    return nil, nil, err
}
```

Figure 4.1: Snapshot creation and post-filter revert in the block production inner function (nitro/arbos/block_processor.go#L477–L515)

The error path at line 505 is safe: if `ApplyTransactionWithResultFilter` returns an error and `Finalise` has not run—either because `ApplyMessage` failed before the result filter, or because the result filter itself failed before `Finalise`)—so the snapshot ID remains valid. However, when `ApplyTransactionWithResultFilter` succeeds, `Finalise` has already executed:

```
result, err = ApplyMessage(evm, msg, gp)
if err != nil {
    return nil, nil, err
}

if resultFilter != nil {
    err = resultFilter(result)
    if err != nil {
        return nil, nil, err
    }
}
// Update the state with pending changes.
var root []byte
if evm.ChainConfig().IsByzantium(blockNumber) {
    evm.StateDB.Finalise(true)
}
// ...
```

Figure 4.2: Execution order in `ApplyTransactionWithEVM` showing `Finalise` runs after the result filter succeeds. (go-ethereum/core/state_processor.go#L158–L172)

`Finalise` calls `clearJournalAndRefund`, which calls `journal.reset()`:

```
func (s *StateDB) Finalise(deleteEmptyObjects bool) {
    // ...
    // Invalidate journal because reverting across transactions is not allowed.
    s.clearJournalAndRefund()
}
```

```
func (s *StateDB) clearJournalAndRefund() {
    s.journal.reset()
    s.refund = 0
}
```

Figure 4.3: `Finalise` invalidates the journal. (go-ethereum/core/state/statedb.go#L848–L886 and go-ethereum/core/state/statedb.go#L1138–L1141)

`journal.reset()` truncates `validRevisions` to length zero:

```
func (j *journal) reset() {
    j.entries = j.entries[:0]
    j.validRevisions = j.validRevisions[:0]
    clear(j.dirties)
    j.nextRevisionId = 0
}
```

Figure 4.4: `journal.reset` wipes all valid revision IDs. (go-ethereum/core/state/journal.go#L77–L82)

When `RevertToSnapshot` is called with the now-invalidated snapshot ID, `revertToSnapshot` performs a binary search over the empty `validRevisions` slice, finds no match, and panics:

```
func (j *journal) revertToSnapshot(revid int, s *StateDB) {
    idx := sort.Search(len(j.validRevisions), func(i int) bool {
        return j.validRevisions[i].id >= revid
    })
    if idx == len(j.validRevisions) || j.validRevisions[idx].id != revid {
        panic(fmt.Errorf("revision id %v cannot be reverted", revid))
    }
    // ...
```

Figure 4.5: `revertToSnapshot` panics when the revision ID is not found. (go-ethereum/core/state/journal.go#L93–L99)

Currently, `extraPostTxFilter` is a no-op that always returns nil, so this code path is never triggered. However, the function is explicitly designed for chain operators to implement custom post-transaction validity checks. Any chain operator who adds a non-trivial implementation that returns an error will cause a node-crashing panic during block production.

Exploit Scenario

A chain operator deploys a custom Arbitrum Orbit chain and implements `extraPostTxFilter` to enforce business-logic constraints, such as rejecting transactions

that exceed a per-address transfer limit. When a transaction exceeds the limit, `extraPostTxFilter` returns an error. The block producer attempts to call `RevertToSnapshot` to undo the transaction's state changes, but because `Finalise` has already wiped the journal, the call panics with "revision id N cannot be reverted." The block producer crashes, halting chain progress until the operator removes the filter or patches the revert logic. An attacker who discovers the filter's rejection criteria can intentionally trigger transactions that cause the filter to reject them, repeatedly crashing the sequencer and halting the chain.

Recommendations

Short term, move the `extraPostTxFilter` call into the `resultFilter` callback passed to `ApplyTransactionWithResultFilter`, placing it after the existing `PostTxFilter` call and before `Finalise` runs. This ensures the journal is still intact when the filter returns an error, and `RevertToSnapshot` at line 505 can safely undo the transaction's state changes.

Long term, add an integration test that verifies `extraPostTxFilter` can reject a transaction without panicking. This test should configure an `extraPostTxFilter` implementation that returns an error for a specific transaction and confirm that the block producer handles the rejection gracefully, discarding the transaction and continuing block production. This prevents regressions if the execution order changes in the future.

5. Stale per-dimension base fees are reused when multi-gas constraints are re-enabled

Severity: **Informational**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-ARBOS60-5
Target: `arbos/l2pricing/l2pricing.go`, `arbos/l2pricing/model.go`, `arbos/l2pricing/multi_gas_fees.go`	

Description

When a chain owner disables multi-gas constraints by setting an empty constraint list, the system stops committing multi-dimensional fee state, but it does not clear the stored per-dimension `current` and `next` base fees held in `MultiGasFees`. `ClearMultiGasConstraints` removes only the constraints themselves.

```
func (ps *L2PricingState) ClearMultiGasConstraints() error {
	length, err := ps.MultiGasConstraintsLength()
	if err != nil {
		return err
	}
	for range length {
		subStorage, err := ps.multiGasConstraints.Pop()
		if err != nil {
			return err
		}
		constraint := OpenMultiGasConstraint(subStorage)
		if err := constraint.Clear(); err != nil {
			return err
		}
	}
	return nil
}
```

Figure 5.1: Clearing constraints does not clear the stored per-dimension fees. (nitro/arbos/l2pricing/l2pricing.go#L316-L332)

`GasModelToUse` selects the multi-gas path the moment the constraint length is greater than zero, so re-enabling constraints switches refund pricing to the multi-gas path immediately within the re-enable block.

```
func (ps *L2PricingState) GasModelToUse() (GasModel, error) {
	if ps.ArbosVersion >= params.ArbosVersion_MultiGasConstraintsVersion {
```

```
                constraintsLength, err := ps.MultiGasConstraintsLength()
                // ...
                if constraintsLength > 0 {
                        return GasModelMultiGasConstraints, nil
                }
        }
        // ...
}
```

Figure 5.2: The multi-gas model becomes active as soon as a constraint exists. (nitro/arbos/l2pricing/model.go#L41-L61)

The fresh per-dimension fees for the new configuration are only written into `next` during the start-of-block pricing update, and they only become `current` after `CommitMultiGasFees` rotates `next` into `current` on a subsequent block. While multi-gas is disabled, `CommitMultiGasFees` returns early and performs no rotation, so the `current` values left over from the previous configuration persist unchanged.

```
func (ps *L2PricingState) CommitMultiGasFees() error {
        gasModel, err := ps.GasModelToUse()
        if err != nil {
                return err
        }
        if gasModel != GasModelMultiGasConstraints {
                return nil
        }
        return ps.multiGasFees.CommitNextToCurrent()
}
```

Figure 5.3: While disabled, no rotation occurs, leaving stale `current` fees in storage. (nitro/arbos/l2pricing/model.go#L396-L405)

As a result, transactions in the re-enable block and the following block compute refunds against per-dimension `current` fees carried over from the previous, possibly outdated, configuration. The stale fees can be higher or lower than the correct values for the new constraints, so users either overpay or underpay relative to the intended multi-dimensional price.

Exploit Scenario

A chain owner disables multi-gas constraints for several blocks and later re-enables them with a new configuration. During the re-enable block and the block that follows, every transaction that computes a multi-dimensional refund reads per-dimension `current` fees left over from the configuration that was active before the multi-gas model was disabled. Because those stale fees do not match the new constraints, refunds are mispriced for two

blocks: where the stale fees are too low, the chain undercharges and loses revenue; where they are too high, users are overcharged.

Recommendations

Short term, clear the stored per-dimension `current` and `next` fees whenever multi-gas constraints are cleared, and recompute or zero them as part of re-enabling constraints so that no transaction reads fees that predate the current configuration. If this tradeoff is acceptable given how rarely this scenario would occur, consider documenting this rare scenario, and monitor the situation when re-enabling the multi-gas constraints.

Long term, for features that can be enabled/disabled, appropriately consider the persistent state changes they make and how they are affected when the feature is re-enabled.

6. The initial-backlog cap can be bypassed by under-supplying gas to the constraint validation

Severity: **Informational**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-ARBOS60-6
Target: `precompiles/ArbOwner.go`, `arbos/l2pricing/model.go`, `arbos/l2pricing/l2pricing.go`	

Description

The `SetMultiGasPricingConstraints` function writes the caller-supplied multi-gas constraints to storage first and then validates them by calling `CalcMultiGasConstraintsExponents` and rejecting the call if any resulting exponent exceeds `MaxPricingExponentBips`. The validation is intended to prevent an owner from installing a constraint whose initial backlog implies a base fee above the intended cap. However, `CalcMultiGasConstraintsExponents` discards the error returned by `MultiGasConstraintsLength`. If the call runs out of gas precisely at that storage read, the length read returns `(0, ErrOutOfGas)`, the dropped error leaves the length as zero, the loop body never executes, and the function returns an all-zero exponent array with a nil error. The cap check in `SetMultiGasPricingConstraints` then passes, and the precompile returns success even though the constraints, including their initial backlog, have already been written to storage.

```
// Calculate exponents for all constraints at once
exps, err := c.State.L2PricingState().CalcMultiGasConstraintsExponents()
if err != nil {
	return fmt.Errorf("failed to calculate multi-gas constraint exponents: %w",
err)
}


// Ensure no exponent exceeds the maximum allowed value
for _, exp := range exps {
	if exp > l2pricing.MaxPricingExponentBips {
		return fmt.Errorf("calculated exponent %d exceeds maximum allowed %d",
exp, l2pricing.MaxPricingExponentBips)
	}
}
```

Figure 6.1: The constraints are already stored before this validation runs. (nitro/precompiles/ArbOwner.go#L647-L658)

```
func (ps *L2PricingState) CalcMultiGasConstraintsExponents()
([multigas.NumResourceKind]arbmath.Bips, error) {
	constraintsLength, _ := ps.MultiGasConstraintsLength()
	var exponentPerKind [multigas.NumResourceKind]arbmath.Bips
	for i := range constraintsLength {
		// ...
	}
	return exponentPerKind, nil
}
```

Figure 6.2: The error from `MultiGasConstraintsLength` is dropped; a zero length skips all validation. (`nitro/arbos/l2pricing/model.go#L275-L328`)

```
func (ps *L2PricingState) MultiGasConstraintsLength() (uint64, error) {
	return ps.multiGasConstraints.Length()
}
```

Figure 6.3: The length read is a storage operation that charges gas and can fail with an out-of-gas error. (`nitro/arbos/l2pricing/l2pricing.go#L281-L283`)

Because the error is discarded rather than propagated, the out-of-gas condition does not abort the precompile. Execution continues, the cap check sees zero constraints, and the owner method succeeds with the over-cap constraints committed to state. This issue requires the chain owner to deliberately craft the gas supplied to the call, so it is reported at informational severity.

Exploit Scenario

A chain owner wants to install a multi-gas constraint whose initial backlog implies a base fee well above the intended cap, which the `MaxPricingExponentBips` check would reject. The owner invokes `SetMultiGasPricingConstraints` with the over-cap constraint and supplies a gas amount tuned so that all of the constraint writes in `AddMultiGasConstraint` succeed but the subsequent `MultiGasConstraintsLength` read inside `CalcMultiGasConstraintsExponents` exhausts the remaining gas. The length read returns zero with a dropped error, the validation loop iterates zero times, the cap check passes, and the precompile commits the over-cap constraint to storage. The chain then prices gas from a backlog the cap was specifically designed to forbid.

Recommendations

Short term, check the error from `MultiGasConstraintsLength` inside `CalcMultiGasConstraintsExponents` instead of discarding it.

Long term, when discarding an error, always ensure that it is truly unnecessary. Where possible, use a pattern of validating data first and then execute the action instead of the opposite order.

7. The base-fee cap exponent delivers ~365x instead of the documented ~5,000x

Severity: **Informational**	Difficulty: **High**
Type: Undefined Behavior	Finding ID: TOB-ARBOS60-7
Target: `arbos/l2pricing/l2pricing.go`, `arbos/l2pricing/model.go`, `util/arbmath/math.go`	

Description

The `MaxPricingExponentBips` constant is intended to cap base-fee growth at approximately 5,000 times the minimum base fee. Its comment states `exp(8.5) ~= x5,000 min base fee`, and the constant is set to 85,000 bips (8.5 when denominated in basis points). However, the base fee is not computed with a true exponential; it is computed with `ApproxExpBasisPoints`, a fourth-degree Maclaurin polynomial. For small inputs, the polynomial closely tracks `e^x`, but it diverges as the input grows. At the cap exponent of 85,000 bips, the polynomial returns a multiplier of about 365x, not the documented 4,915x (the value of `e^8.5`). The cap is therefore roughly 13 times stricter than its own comment claims.

```
// MaxPricingExponentBips caps the basefee growth: exp(8.5) ~= x5,000 min base fee.
const MaxPricingExponentBips = arbmath.Bips(85_000)
```

Figure 7.1: The cap constant and its comment (nitro/arbos/l2pricing/l2pricing.go#L55-L56)

The same approximation is used to translate every computed exponent into a base fee, so the discrepancy applies both to the owner-facing constraint guard (which rejects constraints whose exponent exceeds `MaxPricingExponentBips`) and to the runtime base-fee computation.

```
func (ps *L2PricingState) calcBaseFeeFromExponent(exponent arbmath.Bips) (*big.Int,
error) {
	minBaseFee, err := ps.MinBaseFeeWei()
	if err != nil {
		return nil, err
	}
	if exponent > 0 {
		return arbmath.BigMulByBips(minBaseFee,
arbmath.ApproxExpBasisPoints(exponent, 4)), nil
	} else {
```

```
                return minBaseFee, nil
        }
}
```

Figure 7.2: The base fee uses the polynomial approximation, not a true exponential. (nitro/arbos/l2pricing/model.go#L330-L340)

```
// ApproxExpBasisPoints return the Maclaurin series approximation of e^x, where
// x is denominated in basis points.
// ...
// The quartic polynomial (accuracy = 4) will underestimate e^x by about 5% as
// x approaches 20000 bips.
func ApproxExpBasisPoints(value Bips, accuracy uint64) Bips {
        // ...
        res := b + x/accuracy
        for i := accuracy - 1; i > 0; i-- {
                res = b + SaturatingUMul(res, x)/(i*b)
        }
        // ...
}
```

Figure 7.3: `ApproxExpBasisPoints` is a degree-4 polynomial whose own comment notes a 5% undershoot already at 20,000 bips. (nitro/util/arbmath/math.go#L355-L399)

The effect is that the documented protection level and the implemented protection level diverge. An operator reading the comment will believe the cap permits base-fee escalation up to 5,000x the minimum, while the actual maximum reachable through this exponent is approximately 365x.

Exploit Scenario

A chain operator needs the base fee to be able to rise to 5,000 times the minimum during sustained congestion. The operator configures constraints and relies on `MaxPricingExponentBips` to represent that ceiling. During an actual congestion event, the base fee saturates at 365x the minimum and never approaches the expected 5,000x.

Recommendations

Short term, correct the comment on `MaxPricingExponentBips` so it states the multiplier that `ApproxExpBasisPoints` actually produces at 85,000 bips (approximately 365x), or, if a 5,000x ceiling is the intended behavior, raise the constant to the exponent that yields that multiplier under the polynomial (approximately 176,000 bips).

Long term, expand tests that assert the relationship between `MaxPricingExponentBips` and the resulting base-fee multiplier, and document the accuracy limits of `ApproxExpBasisPoints` at the input ranges the pricing model can actually reach.

8. A SingleDim resource weight is accepted and can under-prices every other dimension

Severity: **Informational**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-ARBOS60-8
Target: `arbos/l2pricing/multi_gas_constraint.go`, `arbos/l2pricing/model.go`	

Description

The `CalcMultiGasConstraintsExponents` function contains a guard that skips the `ResourceKindSingleDim` resource when computing per-dimension exponents, with a comment stating the branch "should never be reached." However, the branch is reachable, because nothing in the constraint-installation path rejects a `SingleDim` weight, and a nonzero `SingleDim` weight corrupts the exponent computation for the legitimate dimensions even though `SingleDim` itself produces no fee.

The validation applied to weight keys is `CheckResourceKind`, which rejects only IDs less than or equal to `ResourceKindUnknown` (0) and greater than or equal to `NumResourceKind` (9). `ResourceKindSingleDim` is ID 6, so it passes validation, and `SetResourceWeights` stores it like any other weight while also folding it into `maxWeight`.

```
func (c *MultiGasConstraint) SetResourceWeights(weights map[uint8]uint64) error {
	for _, kind := range slices.Sorted(maps.Keys(weights)) {
		if _, err := multigas.CheckResourceKind(kind); err != nil {
			return err
		}
	}
	var maxWeight uint64
	for i := range int(multigas.NumResourceKind) {
		// #nosec G115 safe: NumResourceKind < 2^32
		weight := weights[uint8(i)]
		if weight > maxWeight {
			maxWeight = weight
		}
		if err := c.weightedResources[i].Set(weight); err != nil {
			return err
		}
	}
	return c.maxWeight.Set(maxWeight)
}
```

Figure 8.1: A `SingleDim` weight passes `CheckResourceKind` and is stored and counted toward `maxWeight`. (nitro/arbos/l2pricing/multi_gas_constraint.go#L80-L98)

`UsedResources` returns every kind whose stored weight is nonzero, including `SingleDim`, so the exponent loop visits `SingleDim` and hits the `continue`.

```
func (c *MultiGasConstraint) UsedResources() ([]multigas.ResourceKind, error) {
	var result []multigas.ResourceKind
	for i := range uint8(multigas.NumResourceKind) {
		weight, err := c.weightedResources[i].Get()
		// ...
		if weight != 0 {
			result = append(result, multigas.ResourceKind(i))
		}
	}
	return result, nil
}
```

Figure 8.2: `UsedResources` returns `SingleDim` when its weight is nonzero. (nitro/arbos/l2pricing/multi_gas_constraint.go#L181-L193)

```
for _, kind := range usedResources {
	if kind == multigas.ResourceKindSingleDim {
		// The single-dimensional gas dimension shouldn't be used to compute
the base fee.
		// This condition should never be reached but we enforce it just to be
sure.
		continue
	}
	// ...
	divisor := arbmath.SaturatingCastToBips(
		arbmath.SaturatingUMul(uint64(adjustmentWindow),
			arbmath.SaturatingUMul(target, maxWeight)))
```

Figure 8.3: The "should never be reached" guard, and the divisor that depends on `maxWeight` (nitro/arbos/l2pricing/model.go#L299-L324)

The guard prevents `SingleDim` from producing a fee, but it does not prevent `SingleDim` from corrupting the fees of the real dimensions. This happens in two ways: first, `updateBacklog` adds `weight[SingleDim] * posterGas` to the constraint's single shared backlog, which is the dividend for every dimension, so `SingleDim` usage inflates the backlog used to price all dimensions. Second, the per-dimension exponent divisor is `adjustmentWindow * target * maxWeight`, and `maxWeight` is the maximum across all stored weights. A large `SingleDim` weight becomes `maxWeight` and inflates the divisor, shrinking the exponent of every legitimate dimension. The net effect is that a `SingleDim` weight silently under-prices the real dimensions.

```
func (c *MultiGasConstraint) updateBacklog(op BacklogOperation, multiGas
multigas.MultiGas) error {
	totalBacklog, err := c.backlog.Get()
	// ...
	for i := range uint8(multigas.NumResourceKind) {
		weight, err := c.weightedResources[i].Get()
		// ...
		if weight == 0 {
			continue
		}
		resourceAmount := multiGas.Get(multigas.ResourceKind(i))
		weightedAmount := arbmath.SaturatingUMul(resourceAmount,
uint64(weight))
		totalBacklog = applyGasDelta(op, totalBacklog, weightedAmount)
	}
	return c.SetBacklog(totalBacklog)
}
```

Figure 8.4: SingleDim usage with a nonzero weight is added to the shared backlog. (nitro/arbos/l2pricing/multi_gas_constraint.go#L110-L130)

Because installing the SingleDim weight requires a privileged owner action that the system is not expected to take, this is reported at informational severity.

Exploit Scenario

A chain owner installs a multi-gas constraint that assigns a large weight to `ResourceKindSingleDim` alongside the real dimensions. `CheckResourceKind` accepts the `SingleDim` ID, `SetResourceWeights` records it as `maxWeight`, and from then on the per-dimension exponent divisor is dominated by the `SingleDim` weight. Every legitimate dimension is priced with a divisor several times larger than intended, so the base fee for those dimensions rises far more slowly under load than the constraint targets imply.

Recommendations

Short term, reject `ResourceKindSingleDim` in the constraint-installation path so a `SingleDim` weight can never be stored.

Long term, expand the testing to validate that rejection of certain weight keys is implemented correctly. The same happy and unhappy tests should always be implemented when validating data.

9. Saturating multiplication collapses the constraint exponent and bypasses the initial-backlog cap

Severity: **Informational**	Difficulty: **High**
Type: Data Validation	Finding ID: TOB-ARBOS60-9
Target: `arbos/l2pricing/model.go`, `util/arbmath/math.go`, `util/arbmath/bips.go`, `precompiles/ArbOwner.go`	

Description

The `SetMultiGasPricingConstraints` function stores a new constraint and then rejects it if any per-dimension exponent computed by `CalcMultiGasConstraintsExponents` exceeds `MaxPricingExponentBips` (85,000 bips), a guard intended to cap the implied initial base fee. The exponent for a dimension is computed as `10000 * backlog * weight / (adjustmentWindow * target * maxWeight)`. Both the numerator and the denominator are formed with saturating unsigned multiplication (figure 9.1), and large weights cause both sides to saturate to `MaxUint64` independently (figure 9.2). After both are cast toward `MaxInt64`, the division collapses to approximately `MaxInt64 / MaxInt64 = 1` (figure 9.3). As a result, the guard sees an exponent of 1 bip for a constraint whose true exponent is enormous, and accepts it (figure 9.4).

```
divisor := arbmath.SaturatingCastToBips(
	arbmath.SaturatingUMul(uint64(adjustmentWindow),
		arbmath.SaturatingUMul(target, maxWeight)))
// ...
	dividend := arbmath.NaturalToBips(
		arbmath.SaturatingCast[int64](arbmath.SaturatingUMul(backlog, weight)))


	exp := dividend / divisor
```

Figure 9.1: Numerator and denominator each saturate independently before the division. (nitro/arbos/l2pricing/model.go#L299-L322)

```
// SaturatingUMul multiply two integers without over/underflow
func SaturatingUMul[T Unsigned](a, b T) T {
	product := a * b
	if b != 0 && product/b != a {
		product = ^T(0)
	}
```

```
        return product
}
```

Figure 9.2: `SaturatingUMul` clamps any overflow to `MaxUint64`.
(nitro/util/arbmath/math.go#L275-L282)

```
func NaturalToBips(natural int64) Bips {
        return Bips(SaturatingMul(natural, int64(OneInBips)))
}
// ...
func SaturatingCastToBips(value uint64) Bips {
        return Bips(SaturatingCast[int64](value))
}
```

Figure 9.3: Both conversions saturate toward `MaxInt64`, so the ratio collapses to ~1.
(nitro/util/arbmath/bips.go#L18-L63)

```
// Ensure no exponent exceeds the maximum allowed value
for _, exp := range exps {
        if exp > l2pricing.MaxPricingExponentBips {
                return fmt.Errorf("calculated exponent %d exceeds maximum allowed %d",
exp, l2pricing.MaxPricingExponentBips)
        }
}
```

Figure 9.4: The cap check sees the collapsed exponent of 1 and accepts the constraint.
(nitro/precompiles/ArbOwner.go#L653-L658)

Because the over-cap constraint must be supplied by a privileged owner, this is reported as informational severity.

Exploit Scenario

A chain owner constructs a multi-gas constraint with a large weight and a large initial backlog whose true implied exponent is far above the 85,000-bip cap, for example target = 30,000,000, `adjustmentWindow = 1`, `backlog = 800,000,000,000`, and `weight = maxWeight = 1,000,000,000,000`. The owner calls `SetMultiGasPricingConstraints`. During validation, `target * maxWeight` and `backlog * weight` each overflow and saturate, the per-dimension exponent collapses to 1 bip, and the cap check passes even though the correct exponent is roughly 266,666,666 bips. The constraint is committed, and the pricing model proceeds with a backlog the cap was meant to forbid.

Recommendations

Short term, fix the issue appropriately in code by, for example, rejecting constraints whose products can cause an overflow. Alternatively, document this behavior for the chain owners.

Long term, review any use of saturating arithmetic, especially in data validation, to determine if saturation arithmetic is the correct use case, or if overflow should instead be treated as a failure.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

Rating Criteria	
Rating	**Description**
Strong	No issues were found, and the system exceeds industry standards.
Satisfactory	Minor issues were found, but the system is compliant with best practices.
Moderate	Some issues that may affect system safety were found.
Weak	Many issues that affect system safety were found.
Missing	A required component is missing, significantly affecting system safety.
Not Applicable	The category does not apply to this review.
Not Considered	The category was not considered in this review.
Further Investigation Required	Further investigation is required to reach a meaningful conclusion.

B. Code Quality Findings

The following findings are not associated with any specific vulnerabilities. However, fixing them will enhance code readability and may prevent the introduction of vulnerabilities in the future.

- The `SetMaxStylusContractFragments` method of the `ArbOwner` precompile has its `arbosVersion` assigned twice with the same meaning. The first instance is `params.ArbosVersion_60`, and the second instance is `params.ArbosVersion_StylusContractLimit`, which is an alias for ArbOS 60.
- An incorrect comment states that the `adjustmentWindow` is `uint64`, but it is `uint32`.
- The `ExposeMultiGas` option in the default config should be changed to true since the functionality is now fully implemented.
- In `setMultiGasPricingConstraints`, the `maxWeight` calculation could be moved into the previous loop, as demonstrated below:

```
// new !!
                uint256 maxWeight = 0;

                [..]
                for (uint256 j = 0; j < nResources; ++j) {
                    [...]

                    // new !!
                    if (startingBacklogValue > 0) {
                      uint64 weight = constraints[i].resources[j].weight;
                      if (weight > maxWeight) {
                          maxWeight = weight;
                      }
                    }
                }
                if (maxWeight > 0) {
                  // calculate exponents
                }
```

Figure B.1: Example optimization of `setMultiGasPricingConstraints` in `ResourceConstraintManager.sol`

- The `MAX_PRICING_EXPONENT` check should be moved to optimize the function, as demonstrated below:

```
pricingExponents[kind] +=
                            uint64(uint256(startingBacklogValue) * uint256(weight) *
1000 / divisor);
                        if (pricingExponents[kind] > MAX_PRICING_EXPONENT) {
                          // revert
                        }
```

Figure B.2: Example optimization of `setMultiGasPricingConstraints` in `ResourceConstraintManager.sol`

- Coding style is inconsistent across functions in the `ResourceConstraintManager` contract. Some loops use `i++` while others use `++i`.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow @trailofbits on X or LinkedIn and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.